

Reporte de Avance Proyecto Final – Sistemas Operativos

Integrantes del equipo: Fidel Ríos Salazar y
Oswaldo Munoz

Materia: Sistemas Operativos

Nombre Docente: Roberto Carlos Vaquera

Universidad Tecmilenio

Segundo Semestre - Bloque 1

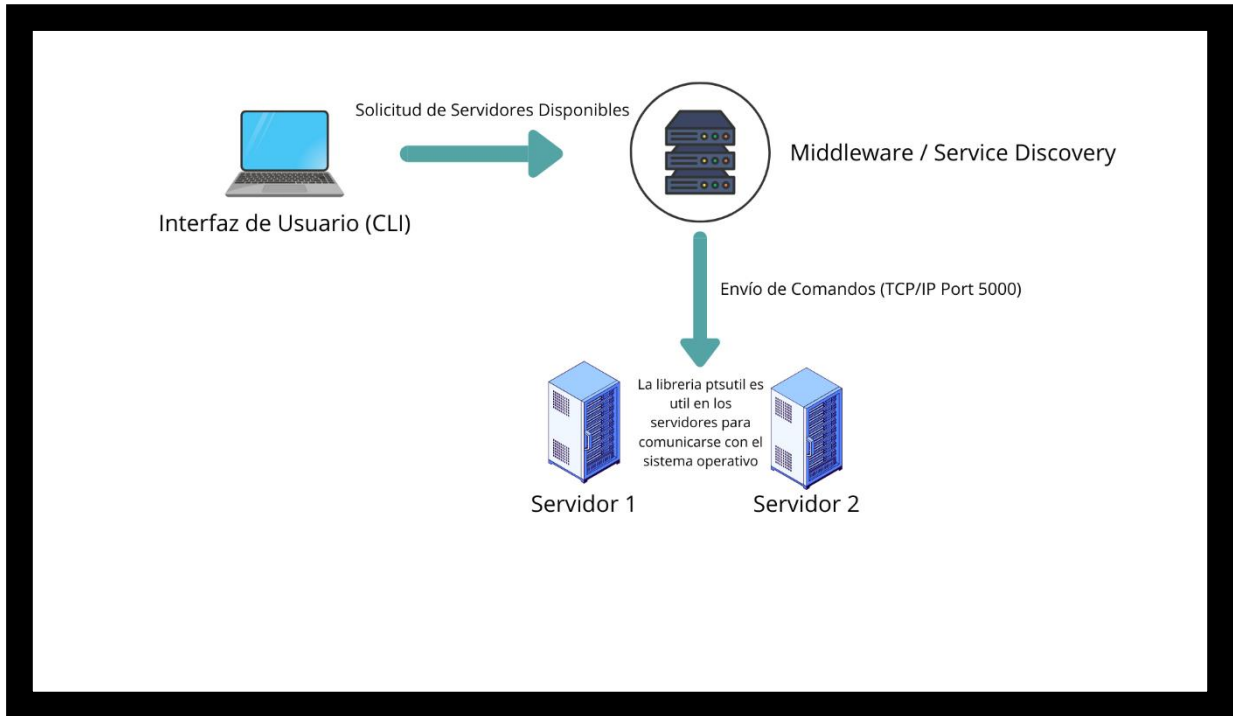
Enlace a repo GitHub: [elfidelito27-
rgb/Proyecto SistemasOperativos](https://github.com/elfidelito27-rgb/Proyecto_SistemasOperativos)

Documentación: Diseño del Sistema

Arquitectura

Se diseñó una estructura de tres capas (Cliente, Middleware y Servidor) para permitir la escalabilidad y el monitoreo de múltiples nodos de red simultáneamente.

Diagrama de Arquitectura:



Explicación del diagrama

El sistema se basa en una arquitectura de red distribuida dividida en tres capas lógicas para asegurar escalabilidad y eficiencia en la gestión de recursos. Como se observa en el siguiente diagrama, la comunicación se centraliza a través de un middleware que facilita el descubrimiento de servicios:

Descripción del flujo

1. Cliente (CLI): Capa de interfaz donde el usuario ingresa comandos.

2. **Middleware:** Actúa como orquestador, recibiendo la solicitud del cliente y redirigiéndola al servidor seleccionado mediante sockets TCP/IP en el puerto 5000.

3.

Servidores (1 y 2): Nodos remotos que ejecutan el agente de monitoreo. Utilizan la librería psutil para obtener datos directamente del sistema operativo y ejecutar acciones de control de procesos.

Decisiones de Diseño:

Protocolo TCP: Se eligió sobre UDP para garantizar que los comandos de administración (como detener un proceso) se entreguen de forma íntegra y sin pérdida de datos.

Librería Psutil: Seleccionada por su alta precisión en el acceso a métricas del Kernel para CPU y Memoria.

Documentación: Implementación (Módulos)

Servidor (server.py): Agente receptor que interactúa con el SO para ejecutar comandos de gestión y enviar estadísticas de hardware.

Cliente (cliente.py): Interfaz de usuario que traduce las entradas de teclado en comandos de red.

Middleware (middleware.py): Capa lógica que gestiona el descubrimiento de servicios y centraliza la comunicación con diversos servidores.

Desafíos enfrentados y soluciones implementadas

Durante el desarrollo de este primer avance, identificamos y resolvimos algunos retos técnicos críticos para asegurar la estabilidad y escalabilidad del sistema:

Desafío 1: Gestión de excepciones en el control de procesos remotos.

Problema: Al intentar manipular procesos mediante comandos como stop, el servidor podía presentar cierres inesperados si el Identificador de Proceso o el PID ya no estaba activo o si el sistema operativo denegaba el acceso por falta de privilegios.

Solución: Se implementó un manejo de errores robusto en el archivo server.py mediante bloques try-except. Estos capturan excepciones específicas de la librería pysqlite, como NoSuchProcess y AccesDenied, permitiendo que el servidor responda con un mensaje de error controlado al cliente sin interrumpir su ejecución.

Desafío 2: Implementación de una arquitectura escalable para múltiples nodos.

Problema: La arquitectura inicial cliente-servidor limitaba la interacción a una única dirección IP estática, lo cual no cumplía con los requisitos de un sistema distribuido capaz de gestionar diversas máquinas simultáneamente.

Solución: Se diseñó y desarrolló un módulo middleware que funciona como un orquestador centralizado. Esta capa facilita el descubrimiento de servicios, permitiendo al cliente seleccionar dinámicamente el servidor con el que desea comunicarse mediante una lista de nodos disponibles, logrando así una gestión distribuida eficiente.

